

Java Übungsaufgaben zum Auffrischen

Abgerundete Summe

Vervollständige die Methode `public int abgerundeteSumme(double a, double b)`, die ihre beiden Parameter addiert und die Summe abgerundet zurück liefert.

Hinweis: `Abgerundet` bedeutet in diesem Kontext, dass der Wert **zur 0 gerundet** wird, also Nachkommastellen wegfallen.

Beispiel 1: <code>a = 13.37;</code>	<code>b = 42.42;</code>	Rückgabewert: 55
Beispiel 2: <code>a = 3.8;</code>	<code>b = 1.5;</code>	Rückgabewert: 5
Beispiel 3: <code>a = -3.8;</code>	<code>b = -1.5;</code>	Rückgabewert: -5

Abgerundete Summe: Lösung

```
public int abgerundeteSumme(double a, double b) {  
    return (int) (a + b);  
}
```

Durchschnitt

Vervollständige die Methode `public double berechneDurchschnitt(double a, double b, double c)`, die den Durchschnitt ihrer drei Double-Parameter liefern soll.

Beispiel: `berechneDurchschnitt(1.5, 1.5, 3.0)`

Rückgabewert: `2.0`

Durchschnitt: Lösung

```
public double berechneDurchschnitt(double a, double b, double c) {  
    final int anzahlSummanden = 3;  
    double summe = a + b + c;  
    return summe / anzahlSummanden;  
}
```

Maximum

Schreibe eine Methode `public int maximum(int a, int b, int c)`, die den größten der drei übergebenen `int`-Werte zurückliefert.

Beispiel: `a = 1; b = 42; c = 21;` Rückgabewert: 42

Maximum: Lösung

```
public int maximum(int a, int b, int c) {  
    int maximum = a;  
  
    if (b > maximum) {  
        maximum = b;  
    }  
    if (c > maximum) {  
        maximum = c;  
    }  
  
    return maximum;  
}
```

Zweidimensionale Punkte

Die vorgegebene Klasse `Punkt` soll Punkte in einem zweidimensionalen Koordinatensystem modellieren.

Erweitere sie um einen Konstruktor, der beide Koordinatenwerte als Parameter erhält und sie geeignet ablegt. Implementiere außerdem zwei sondierende Methoden `double getX()` und `double getY()`, die den jeweiligen Koordinatenwert eines Punktes liefern.

```
class Punkt {  
    private double x;  
  
    private double y;  
  
    // Konstruktor einfügen  
  
    // getX-Methode einfügen  
  
    // getY-Methode einfügen  
}
```


Zweidimensionale Punkte: Lösung

```
class Punkt {  
    private double x;  
    private double y;  
  
    public Punkt(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public double getX() {  
        return x;  
    }  
  
    public double getY() {  
        return y;  
    }  
}
```

Die Rechenoperation Modulo

Schreibe eine Klasse `Taschenrechner`, die das Interface `ArithmetikMix` implementiert.

Hinweis: Wie dir der Titel schon verrät, bietet es sich besonders an, diese Aufgabe mittels Modulo-Operation zu lösen.

```
public interface ArithmetikMix {  
    /**  
     * Berechnet die Summe aller Teiler, die eine Ganzzahl ohne  
     * Rest teilen, wobei nur die positiven Teiler berücksichtigt werden.  
     *  
     * @param n Die zu prüfende Ganzzahl  
     * @return Die Summe der Teiler.  
     */  
    public int divisorSumme(int n);  
}
```

Die Rechenoperation Modulo: Lösung

```
interface ArithmetikMix {  
    int divisorSumme(int n);  
}  
  
class Taschenrechner implements ArithmetikMix {  
    public int divisorSumme(int n) {  
        int zahl = Math.abs(n);  
        int ergebnis = 0;  
        for (int i = 1; i <= zahl; i++) {  
            if ((zahl % i) == 0) {  
                ergebnis += i;  
            }  
        }  
        return ergebnis;  
    }  
}
```

Strings verketten

Schreibe eine öffentliche Methode `verbindeStrings (String s1, String s2)`, die beide Zeichenketten verbindet.

Hierbei soll der kürzere hinter den längeren String gehängt werden. Sind beide gleich lang, so wird `s2` hinter `s1` gehängt. Der neue, zusammengesetzte String soll als Ergebnis geliefert werden.

Wenn `s1` oder `s2` null ist, soll eine `IllegalArgumentException` geworfen werden.

Beispiel: `s1 = "Welt!", s2 = "Hallo "`; Ergebnis: `"Hallo Welt!"`

Strings verketten: Lösung

```
public String verbindeStrings(String s1, String s2) {  
    if (s1 == null || s2 == null) {  
        throw new IllegalArgumentException("Ungültige Strings erhalten.");  
    }  
  
    String ergebnis;  
    if (s1.length() < s2.length()) {  
        ergebnis = s2 + s1;  
    } else {  
        ergebnis = s1 + s2;  
    }  
  
    return ergebnis;  
}
```

Zeichen in einem String finden

Oft muss überprüft werden, ob sich ein gesuchtes Zeichen in einer Zeichenkette befindet. In der Klasse `String` ist bereits eine Methode implementiert, die diese Frage beantwortet.

Schreibe dennoch zur Übung in einer Klasse `ServiceMix` eine öffentliche Methode `findeChar(String, char)`. Sie soll das erste Vorkommen des Zeichens `char` in der Zeichenkette `String` finden und den Index des gefundenen Zeichens als `int` zurückgeben.

Wurde das Zeichen nicht gefunden, so wird `-1` zurückgegeben.

Beispiel: `findeChar("HAW", 'H') = 0`, `findeChar("HAW", 'h') = -1`

Achtung: Die Verwendung von `String.indexOf()` ist nicht erlaubt - das wäre zu einfach!

Zeichen in einem String finden: Lösung

```
public int findeChar(String haystack, char needle) {  
    if (haystack == null) {  
        throw new IllegalArgumentException("Zu durchsuchender String darf nicht null sein");  
    }  
  
    for (int i = 0; i < haystack.length(); i++) {  
        if (haystack.charAt(i) == needle)  
            return i;  
    }  
  
    return -1;  
}
```

Nachfolgebuchstabe

Schreibe eine Methode `public char gibNtenNachfolger(char buchstabe, int n)`, die als Rückgabewert den `n`-ten Nachfolgebuchstaben ausgehend von `buchstabe` liefert.

Beispiel: `buchstabe = 'a'; n = 3;` Rückgabewert: `'d'`

Hinweis: Alle weiteren `char`-Werte nach `'z'` gelten hier ebenfalls als gültige Nachfolgebuchstaben. Negative Werte für `n` können vernachlässigt werden.

Nachfolgebuchstabe: Lösung

```
public char gibNtenNachfolger(char buchstabe, int n) {  
    return (char) (buchstabe + n);  
}
```

Array umkehren

Im Folgenden soll eine öffentliche Methode `reverseArray()` geschrieben werden, welche ein `char`-Array als Parameter erhält. Dieses soll in umgekehrter Reihenfolge als ein `char`-Array wieder zurückgegeben werden.

Beispiel: `['T', 'e', 's', 't']` `['t', 's', 'e', 'T']`

Hinweis: Das übergebene Array darf nicht verändert werden.

Array umkehren: Lösung

```
class ArrayUtil {  
    public char[] reverseArray(char[] input) {  
        char[] reversed = new char[input.length];  
  
        for (int i = 0; i < input.length; i++) {  
            reversed[i] = input[input.length - 1 - i];  
        }  
  
        return reversed;  
    }  
}
```

Array-Summe

Für ein Programm, welches Klausurergebnisse verwaltet, fehlt eine Methode `hatBestanden`.

Die Methode soll für eine Klausur bestimmen, ob der Student diese bestanden hat. Dafür bekommt sie ein Array mit den Punktzahlen der Teilaufgaben und eine Mindestpunktzahl übergeben. Die Methode liefert `true` für bestanden bzw. `false` für nicht bestanden zurück.

Implementiere die Methode `boolean hatBestanden(int[] aufgabenPunkte, int mindestPunkte)`, die obiges Verhalten modelliert.

Array-Summe: Lösung

```
class KlausurService {  
    public boolean hatBestanden(int[] aufgabenPunkte, int mindestPunktZahl) {  
        int summe = 0;  
        for (int punkte : aufgabenPunkte) {  
            summe += punkte;  
        }  
        return (summe >= mindestPunktZahl);  
    }  
}
```